

JavaSubset Compiler $\rightarrow$ x86-64

with Formal Verification (Hoare Logic + Z3)

Jaime Hernandez

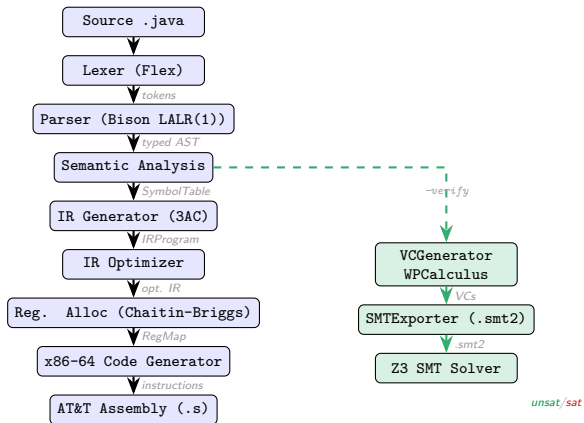
Compiler Design

2026

Table of Contents

- 1 General Architecture
- 2 Front-end: Lexer & Parser
- 3 Semantic Analysis
- 4 Intermediate Representation (3AC)
- 5 x86-64 Backend
- 6 Register Allocation
- 7 Formal Verification
- 8 Tests and Results
- 9 Conclusions

Compiler Pipeline



Language Constructs

- Primitive types: `int`, `double`, `boolean`
- Local variables and static methods
- Operators: arithmetic (`+` `-` `*` `/` `%`), relational (`<` `<=` `>` `>=` `==` `!=`), logical (`&&` `||` `!`)
- Control flow: `if/else`, `while`
- `return`, method calls
- I/O: `System.out.println`, `scanner.nextInt/Double`

Formal Verification Annotations (Phase 5)

<code>@pre expr</code>	Precondition
<code>@post expr</code>	Postcondition
<code>@invariant expr</code>	Loop invariant
<code>@variant expr</code>	Termination function
<code>==></code>	Logical implication
<code>result</code>	Method return value

Full Example

```
@pre x >= 0
@post result >= 0
public static int abs(){...}
```

Lexical Analyzer (Flex)

Main Token Classes

Keywords if, while, return,
int,...

Identifiers [a-zA-Z][a-zA-Z0-9]*

Integers [0-9]+

Reals [0-9]+\.[0-9]+

Operators ==> before ==

Annotations @pre, @post,
@invariant

Order Matters

Flex rules are evaluated in declaration order. "result" must precede the ID

Special Rules – Phase 5 (lexer.l)

```
/* formal verification tokens */
"result"      { return RESULT_TK; }
"@pre"        { return PRE_TK; }
"@post"       { return POST_TK; }
"@invariant"   { return INV_TK; }
"@variant"    { return VAR_TK; }
"=="          { return IMPLIES_TK; }

/* relational operators */
"==" { return EQ_TK; }
"!=" { return NEQ_TK; }
"<=" { return LTE_TK; }
">=" { return GTE_TK; }

/* must come AFTER keywords */
[a-zA-Z][a-zA-Z0-9]* { return ID_TK; }

/* literals */
[0-9]+\.[0-9]+ { return REAL_TK; }
[0-9]+         { return INT_TK; }
```

Syntactic Analyzer (Bison LALR(1))

```
/* Main grammar (parser.y) */
Program : 'public' 'class' ID
        '{' Members '}'

Method  : Annots 'public' 'static'
        Type ID '(' ')' Block

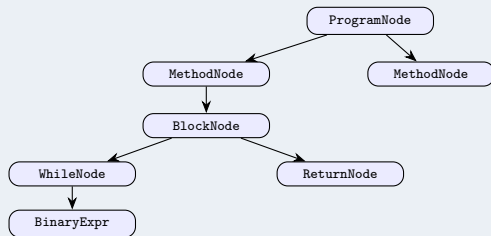
/* Optional formal annotations */
Annots  : /* empty */
        | Annots '@pre' Expr
        | Annots '@post' Expr

Stmt    : Decl | Assign | Return
        | If | PrintStmt
        | 'while' '(' Expr ')'
        OptInv OptVar Block

OptInv  : '@invariant' Expr | /*empty*/
OptVar  : '@variant' Expr | /*empty*/

Expr    : Expr '+' Expr %left 12
        | Expr '-' Expr %left 12
        | Expr '*' Expr %left 13
        | Expr '/' Expr %left 13
        | Expr '==>' Expr %right 5
        | '(' Expr ')'
        | ID | INTLIT | REALLIT
```

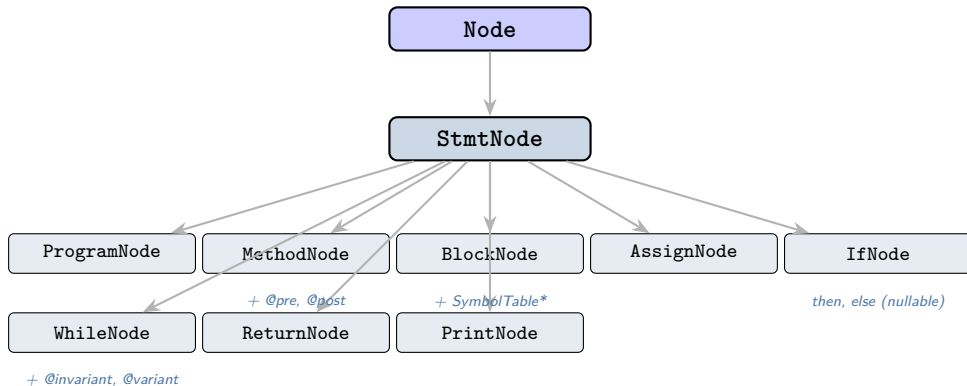
Generated AST



Visitor Pattern (GoF)

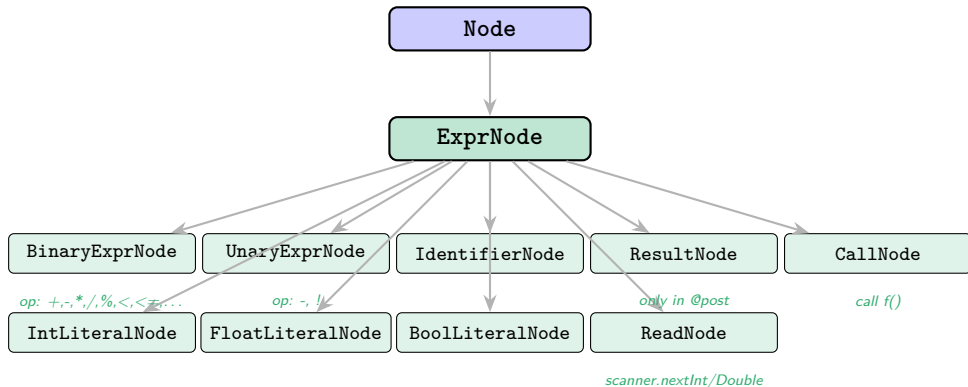
SemanticVisitor, IRBuilder,
ASTVisualizer, DOTVisualizer
– all implement Visitor
without modifying AST nodes

AST Hierarchy – Statement Nodes (StmtNode)



All nodes implement `accept(Visitor*)` – **Visitor pattern (GoF)**

AST Hierarchy – Expression Nodes (ExprNode)



ExprNode carries the type inferred by SemanticVisitor (field type)

Symbol Table and Type Analysis

SymbolTable – Structure

```
struct Symbol {
    string name;
    unsigned type;      // INT=0, REAL=1, BOOL=2
    unsigned addr;      // stack offset (bytes)
    bool isConstant;    // for 'final'
    bool isFunction;
    int returnType;
    // formal annotations
    ExprNode* precondition;
    ExprNode* postcond;
};

class SymbolTable {
    SymbolTable* parent; // parent scope
    map<string, Symbol> table;
public:
    bool set(Symbol s); // false if redecl.
    Symbol* get(string n); // walks scope chain
    SymbolTable* newScope();
};
```

Semantic Checks

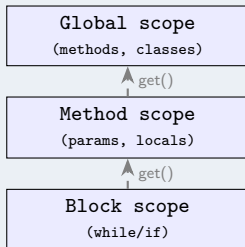
- 1 Declaration before use
- 2 Type compatibility in operations
- 3 No reassignment of final
- 4 Return type matches method signature
- 5 Unreachable code (post-return)
- 6 Valid function calls
- 7 Division by constant zero (warning)

Type Coercion Rules

int op double	⇒	double (IR_ITOR)
int / int	⇒	int (integer division)
bool op bool	⇒	boolean

Hierarchical Scoping and Detected Errors

Scope Chain



Address Assignment

Each variable gets a stack offset:

`int i` → `addr=8`, `int s` → `addr=16`

Temporaries: `addr ≥ 16000` (virtual)

Detected Semantic Errors

```
// sem01: undeclared variable
x = 5; // ERROR: 'x' undeclared

// sem02: type mismatch
int x; x = true; // ERROR: bool -> int

// sem03: redeclaration
int x; int x; // ERROR: already exists

// sem04: wrong return type
public static boolean f() {
    return 42; // ERROR: int != bool
}

// sem05: unreachable code
return 1;
x = 2; // WARNING: dead code
```

Three-Address Code – Design

IRQuad structure:

(op, arg_1, arg_2, res)

Group	Opcodes
Arithmetic	ADD SUB MUL DIV MOD NEG
Logic	AND OR NOT
Comparison	EQ NEQ LT LTE GT GTE
Copy	COPY
Conversion	ITOR RTOI
Control	JMP JZ JNZ LABEL
Functions	FUNC_BEGIN FUNC_END CALL RETURN
I/O	PRINT_INT PRINT_REAL READ_INT READ_REAL

Argument Types (IRArgKind)

ARG_VAR	Variable with stack address
ARG_TEMP	Synthetic temporary tN
ARG_CONST_INT	Integer literal
ARG_CONST_REAL	Real literal
ARG_LABEL	Jump target label
ARG_FUNC	Function name
ARG_NONE	Unused operand (_)

Memory Layout

Variables: offsets 8, 16, 24, ... (real stack)
Temporaries: $\text{addr} \geq 16000$ (virtual)
`newTemp()` assigns address automatically

IR Generation: while + sum Example

Source Code

```
public static int sumToFive() {  
    int i; int s;  
    i = 1; s = 0;  
    while (i <= 5) {  
        s = s + i;  
        i = i + 1;  
    }  
    return s;  
}
```

Generated IR (before optimization)

```
[FUNC_BEGIN] sumToFive  
[COPY]      1          -> i@8  
[COPY]      0          -> s@16  
[LABEL]     L0  
[LTE]       i@8 5      -> t0      ; condition  
[JZ]        t0         -> L1      ; exit if false  
[ADD]       s@16 i@8   -> t1  
[COPY]      t1         -> s@16  
[ADD]       i@8 1      -> t2  
[COPY]      t2         -> i@8  
[JMP]              -> L0  
[LABEL]     L1  
[RETURN]    s@16  
[FUNC_END] sumToFive
```

IR Optimizations: Three Passes

1. Dead Code Elimination

A temporary t is *dead* if defined but never used as an operand.

Iteration to **fixed point**:

$\text{used} \leftarrow \text{all } \text{arg1}, \text{arg2}$

Remove quads with $\text{result} \notin \text{used}$

Repeat until no change

Not removed: JMP, LABEL, RETURN, PRINT, CALL (side effects)

2. Constant Folding

If arg_1 and arg_2 are literals, evaluate at compile time:

ADD 3 4 $\rightarrow$ COPY 7

MUL 2 5 $\rightarrow$ COPY 10

LTE 3 5 $\rightarrow$ COPY 1

DIV x 0 $\rightarrow$ error

Eliminates computations independent of variables.

3. Copy Propagation

If $\text{COPY } a \rightarrow t_i$ exists, replace uses of t_i with a :

COPY $s \rightarrow t_1$

ADD $t_1 \ i \rightarrow t_2 \implies$

ADD $s \ i \rightarrow t_2$

Then t_1 becomes dead and pass 1 removes it.

Combined result: -31% instructions in reference tests

Optimization Effect: Before and After

Unoptimized IR (12 quads)

```
[COPY] 1      -> t0    ; const
[COPY] t0     -> i@8   ; copy prop
[COPY] 0      -> t1    ; const
[COPY] t1     -> s@16  ; copy prop
[LABEL] L0
[LTE] i@8 5    -> t2
[JZ] t2       -> L1
[ADD] s@16 i@8 -> t3
[COPY] t3     -> s@16
[ADD] i@8 1    -> t4
[COPY] t4     -> i@8
[JMP]        -> L0
```

Optimized IR (8 quads, -33%)

```
; t0, t1 removed (copy propagation)
[COPY] 1      -> i@8
[COPY] 0      -> s@16
[LABEL] L0
[LTE] i@8 5    -> t2
[JZ] t2       -> L1
[ADD] s@16 i@8 -> s@16 ; direct
[ADD] i@8 1    -> i@8  ; direct
[JMP]        -> L0
; t3, t4 removed (dead code)
```

Optimization Pipeline

DeadCode → **ConstFold** → **CopyProp** → **DeadCode** (2nd pass cleans up after CopyProp)

x86-64 Code Generation

System V AMD64 ABI

Arguments %rdi, %rsi, %rdx, %rcx, %r8, %r9

Int return %rax

Double return %xmm0

Caller-saved %rcx, %rdx, %r8-%r11

Callee-saved %rbx, %r12-%r15, %rbp

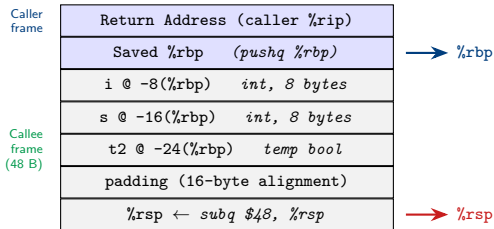
Alignment %rsp multiple of 16 before call

```
/* standard prologue / epilogue */
pushq %rbp
movq %rsp, %rbp
subq $N, %rsp ; N aligned to 16 bytes
/* ... body ... */
movq %rbp, %rsp
popq %rbp
ret
```

Compiled Loop – sumToFive

```
sumToFive:
    pushq %rbp
    movq %rsp, %rbp
    subq $48, %rsp ; 3 vars x 8 bytes
    movq $1, -8(%rbp) ; i = 1
    movq $0, -16(%rbp) ; s = 0
.L0:
    movq -8(%rbp), %rax ; i
    cmpq $5, %rax
    setle %al
    movzbl %al, %rax ; t2 = (i<=5)
    movq %rax, -24(%rbp)
    testq %rax, %rax
    je .L1 ; JZ
    movq -16(%rbp), %rax ; s
    addq -8(%rbp), %rax ; s + i
    movq %rax, -16(%rbp) ; s
    movq -8(%rbp), %rax ; i
    addq $1, %rax
    movq %rax, -8(%rbp) ; i
    jmp .L0
.L1:
    movq -16(%rbp), %rax ; return s
    movq %rbp, %rsp
```

Stack Frame Layout



Frame Size Rules

- Each variable/temp: 8 bytes (int or double)
- Total size: $\lceil N \times 8/16 \rceil \times 16$
- Example: 3 vars → 24 B → 48 B (aligned)

Variable Access

```
movq -8(%rbp), %rax    read i
movq %rax, -16(%rbp)   write s
```

Offset = addr × (-1) from %rbp

ABI Invariant

%rsp must be a multiple of 16
immediately before any call

double Support: SSE2 Instructions

SSE2 Registers and Operations

Registers: %xmm0–%xmm15 (128-bit)
Move: movsd (scalar double)
Arithmetic: addsd, subsd, mulsd, divsd
Compare: comisd + setle/setlt/...
Conversion: cvtsi2sd (int→double)
 cvttsd2si (double→int)
Return: %xmm0

double Constants on Stack

double cannot be loaded with movq \$lit. Stored in .rodata or converted via cvtsi2sd when derived from an integer.

Example: $x = 3.14 * 2.0$

```
; IR_ITOR: int -> double
cvtsi2sd %rax, %xmm1

; double arithmetic
movsd -8(%rbp), %xmm0 ; x (double)
mulsd -16(%rbp), %xmm0 ; x * y
movsd %xmm0, -24(%rbp) ; result

; double comparison: x < 0.0
xorpd %xmm1, %xmm1 ; 0.0
comisd %xmm1, %xmm0 ; cmp x, 0.0
setb %al
movzbq %al, %rax

; double return
movsd -8(%rbp), %xmm0
ret
```

Three Phases of the Algorithm

1 Liveness Analysis

Fixed-point equations per basic block.
Determines which variables are *live* at each point.

2 Interference Graph

Edge (d, v) if variable d is defined while v is live.

Two variables with an edge cannot share a register.

3 Graph Coloring (K -coloring)

Simplify $\rightarrow$ *Spill* $\rightarrow$ *Select*

Each color = one physical register.

Available Registers ($K = 6$)

Type	Registers (caller-saved)
Integers	%rcx, %rdx, %r8 %r9, %r10, %r11
Doubles	%xmm2, %xmm3, %xmm4 %xmm5, %xmm6, %xmm7

Variables that get no register $\Rightarrow$ **spill** (stack)

Chaitin-Briggs: Phase 1 – Liveness Analysis

Fixed-Point Equations per Basic Block B

$\text{use}[B] = \text{vars used before being defined in } B$

$\text{def}[B] = \text{vars defined in } B$

$\text{live_in}[B] = \text{use}[B] \cup (\text{live_out}[B] \setminus \text{def}[B])$

$\text{live_out}[B] = \bigcup_{S \in \text{succ}(B)} \text{live_in}[S]$

Backward iteration (successors to predecessors)

until no $\text{live_in}/\text{live_out}$ changes (**fixed**

Example: Loop Block

Instruction	use	def
LTE i 5 -> t0	i	$t0$
JZ t0 -> L1	$t0$	—
ADD s i -> t1	s, i	$t1$
COPY t1 -> s	$t1$	s
ADD i 1 -> t2	i	$t2$
COPY t2 -> i	$t2$	i
JMP L0	—	—

live_out of LTE instruction:
 $\{i, s, t0\}$

Key Property

If d is defined with $v \in \text{live_out}$
 $\Rightarrow \text{edge}(d, v)$ in the interference graph

Chaitin-Briggs: Interference Graph and Coloring

Analyzed IR Fragment

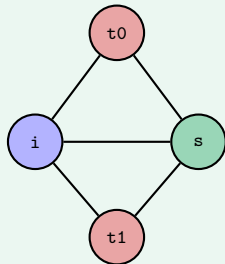
Quad	def	live_out
ADD i 1 -> t0	t0	{i, s, t0}
COPY t0 -> i	i	{i, s}
ADD s i -> t1	t1	{i, s, t1}
COPY t1 -> s	s	{i, s}
RETURN s	-	$\emptyset$

Interference edges:

$t0 \leftrightarrow i$, $t0 \leftrightarrow s$

$t1 \leftrightarrow i$, $t1 \leftrightarrow s$

Graph + Coloring ($K = 3$ illustrative)



$i \rightarrow \%rcx$ $s \rightarrow \%rdx$

$t0, t1 \rightarrow \%r8$ (no interference between them)

Result with Spill

```
$ ./compiler --reg-alloc t04.java
```

Hoare Logic: Triple and Verification Method

Hoare Triple

$$\{P\} S \{Q\}$$

If precondition P holds *before* executing S ,
and execution *terminates*, then postcondition Q holds *after*.

Concrete Example

$\{true\} x := -5; \text{ if}(x < 0) \text{ ret } -x \{result \geq 0\}$

Trivial precondition, postcondition: non-negative

Main Condition to Verify

$$P \Rightarrow WP(body, Q)$$

valid $\iff$ Z3 returns unsat to

$$\neg(P \Rightarrow WP(body, Q))$$

Method Rule

For @pre/@post annotations:

$$VC_{\text{corr}} = P \Rightarrow WP(body, Q)$$

Generated automatically by VCGenerator.

WP Calculus: Rules (1/2) – Assignment, Sequence, Return

Assignment

$$\text{WP}(x := e, Q) = Q[e/x]$$

Textual substitution of x with e in Q .

Example – Assignment

$$\text{WP}(x := 3, x > 0) = (3 > 0) = \text{true}$$

Sequence

$$\text{WP}(S_1 ; S_2, Q) = \text{WP}(S_1, \text{WP}(S_2, Q))$$

Backward composition: computed from the end.

Example – Sequence

$$\begin{aligned} \text{WP}(x := 3 ; y := x + 1, y > 0) \\ &= \text{WP}(x := 3, \text{WP}(y := x + 1, y > 0)) \\ &= \text{WP}(x := 3, (x + 1) > 0) \\ &= (3 + 1) > 0 = \text{true} \end{aligned}$$

WP Calculus: Rules (2/2) – Conditional and Loop

Conditional

$$\begin{aligned} \text{WP}(\text{if}(B) S_1 \text{ else } S_2, Q) \\ &= (B \Rightarrow \text{WP}(S_1, Q)) \\ &\quad \wedge (\neg B \Rightarrow \text{WP}(S_2, Q)) \end{aligned}$$

Each branch must guarantee Q under its entry condition.

Example – Conditional

$$\begin{aligned} Q &= \text{result} \geq 0, \quad B = (x < 0) \\ \text{WP}(\text{if}(x < 0) \{\text{ret } -x\} \text{ else } \{\text{ret } x\}, Q) \\ &= (x < 0 \Rightarrow -x \geq 0) \\ &\quad \wedge (x \geq 0 \Rightarrow x \geq 0) \end{aligned}$$

Loop with Invariant I

$$\text{WP}(\text{while}_I(B) \{S\}, Q) = I$$

Why the Loop Returns I

The compiler cannot compute the WP of a loop without knowing how many iterations it will run. The invariant I acts as a *summary* of

Atomic Predicates (Predicate.h)

Abstract Base Class Predicate

The entire WP logical algebra is built on this interface with **4 pure virtual operations**:

<code>subst(var, expr)</code>	substitution $P[e/x]$
<code>toSMT()</code>	SMT-Lib2 prefix emit
<code>toString()</code>	human-readable form
<code>clone()</code>	deep copy

Atomic Predicates – 6 Classes

Class	Repr.	SMT
TruePred	true	true
FalsePred	false	false
VarPred	x	x
ResultPred	result	result
IntLitPred	42	42
RealLitPred	3.14	3.14

Substitution rule in VarPred:

```
subst("x", e) returns
  e->clone()   if name == "x"
  new VarPred(name) otherwise
```

`fromExpr(ExprNode*)` converts AST nodes to predicates – bridge between parser and

Compound Predicates and Recursive Substitution

Logical and Arithmetic Connectives – 6 Classes

Class	Repr.	SMT-Lib2
BinaryPred	$(a \text{ op } b)$	<code>(op a b)</code>
AndPred	$(P \wedge Q)$	<code>(and P Q)</code>
OrPred	$(P \vee Q)$	<code>(or P Q)</code>
NotPred	$(\neg P)$	<code>(not P)</code>
ImpliesPred	$(P \Rightarrow Q)$	<code>(=> P Q)</code>
NegPred	$(\neg P)$	<code>(- P)</code>

Substitution propagated recursively:

```
AndPred::subst(v,e)
  → new AndPred(left->subst(v,e),
    right->subst(v,e))
```

Example: $WP(x := -5, x \geq 0)$

Initial predicate Q :

```
BinaryPred(">=", VarPred("x"),
IntLitPred(0))
```

```
subst("x", IntLitPred(-5)):
```

```
BinaryPred(">=", IntLitPred(-5),
IntLitPred(0))
```

```
toString(): (-5 ≥ 0)
```

```
toSMT(): (>= (- 5) 0)
```

Z3 evaluates: *false* → sat **[BUG]**

ImpliesPred in SMT

WP Derivation (1/2): Code and Objective

Function to Verify

```
1 @pre  true
2 @post result >= 0
3 public static int absValue() {
4     int x;
5     x = -5;           // S1
6     if (x < 0)         // S2
7         return -x;    // S2a:
8     then branch
9     return x;         // S3: else
10    branch
11 }
```

Objective: prove the VC

$$\underbrace{true}_P \Rightarrow \text{WP}(\text{body}, \underbrace{\text{result} \geq 0}_Q)$$

Body Structure (CPS)

Processing is **forward** (top to bottom),
passing the continuation explicitly.

Analysis order: $S1 \rightarrow S2 \rightarrow \{S2a \parallel S3\}$

WP Derivation (2/2): Applying the Rules

Target postcondition: $Q = \text{result} \geq 0$

Step 1 – Return rule:

$$\text{WP}(\text{ret } x, Q) = Q[x/\text{result}] = (x \geq 0)$$

$$\text{WP}(\text{ret } -x, Q) = Q[-x/\text{result}] = (-x \geq 0)$$

Step 2 – Conditional rule ($B = x < 0$):

$$\text{WP}(S2, Q) = (x < 0) \Rightarrow (-x \geq 0)$$

$$\wedge (x \geq 0) \Rightarrow (x \geq 0)$$

Step 3 – Assignment rule $x := -5$:

$$\text{WP}(x := -5, \text{WP}(S2, Q)) = \text{WP}(S2, Q)[-5/x]$$

$$= (-5 < 0) \Rightarrow (-(-5) \geq 0)$$

$$= \text{true} \Rightarrow \text{true} = \text{true}$$

$$\wedge (-5 \geq 0) \Rightarrow (-5 \geq 0)$$

$$= \text{false} \Rightarrow \text{false} = \text{true}$$

The CPS Problem: Early Returns

Naive Rule: Backward Composition Fails

For body `if(x<0) {ret -x;} ret x;`:

The standard rule $WP(S_1, WP(S_2, Q))$ processes

`ret x (=  $S_2$ )` first $\rightarrow Q[x/\text{result}]$,
then the `if` with that postcondition.

But the `then` branch executes `ret -x` and **exits**: S_2 must not run after it. Backward composition silently ignores this.

Solution: wpSeq Forward with Continuation

```
Predicate* wpSeq(stmts, idx, Q) {
    auto* s = stmts[idx];

    // return: applies Q and DISCARDS the rest
    if (auto* r = cast<ReturnNode>(s))
        return wpReturn(r, Q); // Q[e/
                                // result]

    // if: forks continuation into both branches
    if (auto* i = cast<IfNode>(s)) {
        auto cont = stmts[idx+1 .. end];
        Predicate* B = fromExpr(i->cond);
        return new AndPred(
            new ImpliesPred(B->clone(),
                wpBranchAndCont(i->then, cont, Q
                )),
```

Loop VCs: The Four Conditions

For `while(B){S}` with invariant I and variant V :

VC_1 – Initiation

$$P \Rightarrow \text{WP}(S_{\text{preloop}}, I)$$

The precondition implies the invariant holds *on entry*.

VC_3 – Use of Invariant

$$(I \wedge \neg B) \Rightarrow \text{WP}(S_{\text{post}}, Q)$$

On exit ($\neg B$ holds), the invariant implies the postcondition.

VC_2 – Preservation

$$(I \wedge B) \Rightarrow \text{WP}(S_{\text{body}}, I)$$

If the invariant holds and the guard is true, it still holds after the body.

VC_4 – Termination (@variant)

$$(I \wedge B) \Rightarrow \text{WP}(S_{\text{body}}, 0 \leq V' < V)$$

The variant V decreases strictly and stays non-negative.

Loop VCs: Strong Invariant and Variant

Weak Invariant $\Rightarrow$ VC Fails

Loop: `while(i<=5){s=s+i; i=i+1;}`

With $I = s \geq 0$:

$VC_2: (s \geq 0 \wedge i \leq 5) \Rightarrow (s + i) \geq 0$

Z3 finds counterexample: $i = -3, s = 0$
 $\Rightarrow s + i = -3 < 0$ **sat [BUG]**

The invariant does not capture that $i \geq 1$.

Correct Invariant + Variant

With $I = (s \geq 0) \wedge (i \geq 1)$:

$VC_2: (s \geq 0 \wedge i \geq 1 \wedge i \leq 5) \Rightarrow (s + i) \geq 0$

Z3: **unsat [OK]**

Variant: `@variant 6 - i`

$VC_4: (I \wedge i \leq 5) \Rightarrow$

$$0 \leq (6 - (i + 1)) < (6 - i)$$

$$= (5 - i \geq 0) \wedge (5 - i < 6 - i) = \text{true} \text{ [OK]}$$

SMT-Lib2: Structure of the Generated File

Flow: VCGenerator $\rightarrow$ SMTEncoder $\rightarrow$ Z3

```
1 ; == absValue: correctness ==
2 (push 1)
3 (declare-fun x () Int)
4 ; Negation of VC: P => WP(body, Q)
5 (assert (not
6   (=> true          ; @pre P
7     (and
8       ; then: x<0 => -(-5)>=0
9       (=> (< (- 5) 0)
10         (>= (- (- 5)) 0))
11       ; else: x>=0 => -5>=0
12       (=> (not (< (- 5) 0))
13         (>= (- 5) 0))
14     )
15   )
16 ))
17 (check-sat)      ; unsat -> VC valid [OK]
18 (pop 1)
```

General Structure of Each VC

1. (push 1) – isolated context
2. declare-fun – free variables
3. (assert (not VC)) – negation
4. (check-sat) – Z3 responds
5. (pop 1) – clears context

Why the VC is Negated

If $\neg VC$ is **unsatisfiable** (unsat),
no assignment can refute VC
 $\Rightarrow VC$ is valid for all inputs. **[OK]**

Types of VCs Generated by VCGenerator

Verification Condition Catalog

#	VC	Trigger
1	Method correctness	@pre/@post
2	Invariant initiation	@invariant
3	Invariant preservation	@invariant
4	Invariant use	@invariant
5	Termination	@variant
6	Division-by-zero safety	operator /

Z3 Responses

Response		Meaning
unsat	[OK]	VC valid
sat	[BUG]	Bug / error
unknown		Timeout

Completeness

For methods *without* loops, the WP is **decidable** in linear integer arithmetic (LIA).
Loops require a *manual* invariant: no automatic inference is implemented.

Safety: Automatic Division-by-Zero VC

Without Precondition [BUG]

```
@pre  true
@post result >= 0
public static int divTest() {
    int x; int y;
    x = 10;
    y = 0;           // <-- zero
    divisor!
    return x / y;
}
```

Generated VC:

$$true \Rightarrow y \neq 0$$

With Adequate Precondition [OK]

```
@pre  y != 0
@post result >= 0
public static int safeDiv() {
    int x; int y;
    ...
    return x / y; // safe
}
```

Generated VC:

$$(y \neq 0) \Rightarrow y \neq 0$$

Trivially valid.

Demo: Three Verified Methods

```
1 @pre true
2 @post result >= 0
3 public static int absValue() {
4     int x; x = -5;
5     if (x < 0) return -x;
6     return x;
7 }
```

```
1 @post result > 10 // intentional BUG!
2 public static int wrongMethod() {
3     int x; x = 5; return x;
4 }
```

```
1 @post result >= 0
2 public static int sumLoop() {
3     int i; int s; i=1; s=0;
4     while (i <= 5)
5         @invariant s>=0 && i>=1
6         @invariant 6>=i
```

Verifier Output

```
$ ./compiler --verify demo.java
```

```
[OK] absValue
      correctness (unsat)
```

```
[OK] sumLoop
      initiation (unsat)
```

```
[OK] sumLoop
      preservation (unsat)
```

```
[OK] sumLoop
      use (unsat)
```

```
[OK] sumLoop
      correctness (unsat)
```

```
[OK] sumLoop
      termination (unsat)
```

```
[BUG] wrongMethod
      correctness (sat)
```

Test Suite: 11/11 – Breakdown

Backend End-to-End (5)		Semantic – Errors (3)		Formal Verification (3)	
t01_print	PASS	sem01_undecl	PASS	ver01_simple	PASS
t02_arith	PASS	sem02_type	PASS	ver02_inv	PASS
t03_if_else	PASS	sem03_redecl	PASS	demo_verified	PASS
t04_while	PASS	Verify the compiler rejects ill-typed code		Check OK detection and BUG detection	
t05_funcs	PASS				

Each test: compile → assemble →
execute → compare stdout

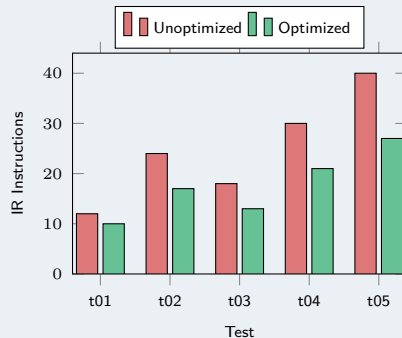
`make && tests/run_tests.sh ⇒ 11 / 11 passed`

Compiler Metrics

Project Size

Component	Lines (approx.)
Lexer (<code>lexer.l</code>)	200
Parser (<code>parser.y</code>)	600
AST Nodes (<code>ASTNodes.h</code>)	170
Semantic (<code>SemanticVisitor</code>)	450
IR + Optimizer	380
x86-64 Backend	650
Reg. Allocator (Chaitin-Briggs)	310
WPCalculus + VCGenerator	500
SMTExporter + Predicates	350
Total	≈3,600

IR Reduction by Optimization



Average improvement: -31% IR instructions

Dead code elim. + const. fold + copy prop.

Phase Summary and Milestones

Phase	Milestone	Key Components	
1	Front-end	Flex + Bison LALR(1), C++ AST, Visitor	✓
2	Semantic	SymbolTable, types, coercion, scoping	✓
3	IR + Optim.	3AC quads, dead code, folding, copy	✓
4	Backend	x86-64 AT&T, SysV ABI, SSE2, frame	✓
4b	Reg. Alloc.	Chaitin-Briggs, liveness, interference	✓
5	Verification	Hoare, WP-CPS, VCGenerator, Z3, SMT	✓

Main Technical Contributions

- Full end-to-end compiler: Java $\rightarrow$ x86-64 assembly
- WP calculus with continuations (CPS) for early returns in if/else
- Automatic VCs: correctness, loop invariants, termination, div/0 safety

Future Work

- Automatic invariant inference (Daikon / abstract interpretation)
- Array support with separation logic
- SSA form for advanced optimizations (GVN, LICM)
- LLVM-IR backend as alternative to direct

Thank You

JavaSubset Compiler $\rightarrow$ x86-64
with Formal Verification via Hoare Logic + Z3

<code>make && tests/run_tests.sh</code>	$\Rightarrow$ 11/11 passed
<code>./compiler -verify demo.java</code>	$\Rightarrow$ 6 OK, 1 BUG detected
<code>./compiler -reg-alloc t04.java</code>	$\Rightarrow$ Chaitin-Briggs coloring
<code>pdflatex presentation.tex</code>	$\Rightarrow$ these slides